

30 | JavaScript 引擎：双向通讯底层原理是什么？

2022-11-1 蒋宏伟



你好，我是蒋宏伟。

在上一讲中，我们介绍了 React Native 使用 Bridge、Turbo Modules、Fabric Components 实现了 JavaScript 和 Native 之间的双向通讯，而双向通讯离不开我今天要和你介绍的 JavaScript 引擎。

常见的 JavaScript 引擎有 V8 和 JavaScriptCore。V8 引擎是 Google 出品的，Chrome 浏览器用的就是它。而 JavaScriptCore 引擎是苹果主导的，它是 Safari 浏览器的引擎。在浏览器下载完成 JavaScript 脚本后，引擎开始执行这些字符串形式的脚本，这些字符串脚本最终会转换机械码，并在硬件上执行。这就是浏览器中页面能够跑起来的原因。

而在 React Native 新架构中，它用的引擎是 Facebook 团队自己研制的 Hermes 引擎。Hermes 引擎和常见的 V8 或 JavaScriptCore 的功能几乎一样，不同的是 Hermes 是为移动端定制的，在启动性能上有不少优势。类似浏览器中的页面，React Native 要跑起来，也离不开 JavaScript 引擎的支持。不仅如此，引擎还是 JavaScript 和 Native 双向通讯的秘密所在。

今天，我们就来揭开这个秘密。

宿主对象

在开始揭秘之前，我先和你介绍一下宿主对象，它是通讯的关键：

那宿主对象是什么呢？我们来看三个例子：

```
// ①
var a = 1+1
// ②
var div = document.createElement('div')
// ③
var view = nativeFabricUIManage.createNode('View')
```

第一个例子，是一段代码 `var a = 1+1` 。

这段代码只有最基础的语法，非常简单。无论在 React Native 中，在浏览器中，甚至是 node.js 中，又无论底层用的是 V8 引擎，JavaScript Core 引擎，还是 Hermes 引擎，它都能正常地执行。

你可能会疑问，这不很正常吗？它就应该正常运行啊。

但你反过来再想，为什么所有的 JavaScript 引擎都会支持 `var a = 1+1` 这个语法呢？它们难道是事先串通好的？

是的，所有的 JavaScript 引擎遵循了同样的标准。在 `var a = 1+1` 这段代码中，数字 1、操作符号 +、定义变量的方法 var，都是 JavaScript 引擎自身提供的。

但无论是哪个科技巨头主导的，它设计的 JavaScript 引擎，都必须遵照 ECMAScript 标准进行实现。而数字、操作符号、定义变量的方法，以及引擎要内置哪些对象等等，这些都在标准中有明确定义，因此，`var a = 1+1` 这段代码无论在哪个引擎上都能正常执行。

然而，并不是所有的 JavaScript 语法，都在 ECMAScript 标准中有定义，我们再来看两个例子。

第二个例子是 `var div = document.createElement('div')`，第三个例子是 `var view = nativeFabricUIManage.createNode('View')`。

很明显，`var div =` 这段代码，哪个 JavaScript 引擎都必须认识，但是 `document` 和 `nativeFabricUIManage` 就不是哪个引擎都认识了。

更准确地说，只有浏览器中的 JavaScript 引擎，认识 `document`，也只有 React Native 中的引擎，认识 `nativeFabricUIManage`。`document` 和 `nativeFabricUIManage` 和引擎无关，只和它所在的平台相关。例如，同样是 JavaScriptCore 引擎，但只有在 React Native 中时，它才认识 `nativeFabricUIManage`，而在 Safari 中时就不认识了。

这些和引擎无关的，只和所在平台环境相关的对象，是什么呢？

它们就是宿主对象。这些由宿主环境自行创建的，而不是 ECMAScript 标准定义的对象，就是宿主对象。

具体地说，浏览器中实现 `document` 对象，遵循的是和浏览器相关的 [W3C 标准](#)，不是 ECMAScript 语言标准，因此它只在浏览器宿主环境中存在。而 React Native 中的 `nativeFabricUIManage` 对象，是 React Native 团队为了实现双向通讯而创建的对象，只在 React Native 中存在。它们不是 JavaScript 语言的一部分，而是其宿主环境的一部分。

这节课，咱们要了解双向通讯的底层，就要去探究 JavaScript 引擎是如何利用这些宿主对象，实现双向通信的。

把玩引擎

双向通信的重点，不在于 C++ 与 Java 或 Objective-C 之间的通信，而是 C++ 与 JavaScript 之间的通信。熟悉 Native 的同学都清楚，C++ 与 Java 之间的通信，早有成熟的方案，叫做 JNI，而 C++ 和 Objective-C 是同一类语言，它们之间本来就可以直接调用。

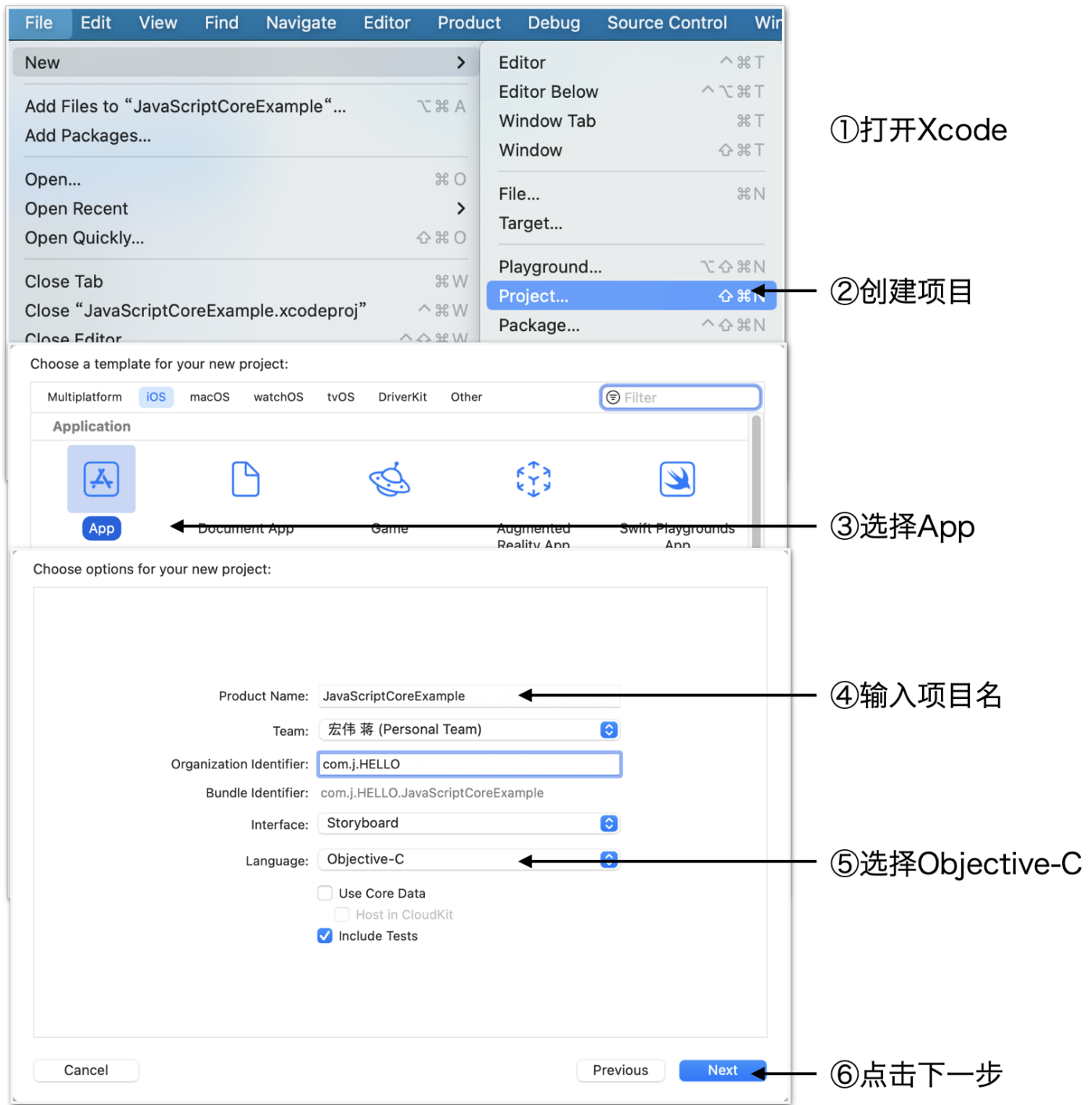
因此，我们探究双向通信的重点在于，**React Native 是如何实现 C++ 与 JavaScript 之间的通信的。**

按道理说，我应该用 Hermes 引擎和你介绍 C++ 与 JavaScript 之间是如何实现通信，这样会更贴近咱们新架构的主题。然而，比较可惜的是，Hermes 根本没有接口文档，并不方便我们学习。因此，我用的是 JavaScriptCore。

但你完全不用太担心，这会影响你对新架构的学习，因为，虽然 JavaScriptCore 和 Herems 提供的接口名字不一样，但二者的功能是几乎一样的，而且 JavaScriptCore 引擎的文档更健全，便于你学习和理解。

为了加深你的理解，咱们一起动手，把玩一下引擎，亲手实现一个简单的、支持双向通信的加减法函数吧。

接下来，我们会用到 Xcode 创建一个 Objective-C 项目。如果你和我一样是前端同学，没有客户端背景也没关系，紧跟步子，一步一步地来，相关步骤的截图，我都放在下面了：



如上图所示，使用 Xcode 新建一个项目，该项目是个 App 项目，它的名字是 JavaScriptCoreExample，开发语言上选择 Objective-C，点击下一步，这样你就把项目创建好了。

一个新项目的第一行代码必须是什么？必须是 hello world，是吧？



①请你首先打开，入口文件 AppDelegate.m 文件，②在文件第二行代码中使用 `import <JavaScriptCore/JavaScriptCore.h>` 引入引擎。

③然后，找到 `application` 的 `didFinishLaunchingWithOptions` 的启动完成回调函数，④使用 Objective-C 语法创建字符串 `console.log('hello world')`，这就是我们要执行的第一行代码。

⑤接着，我们初始化引擎暴露上下文 `JSContext` 类，生成一个 `context` 对象，⑥在 `context` 对象上有一个 `evaluateScript` 方法，它会执行我们指定的脚本 `hello world`。

这就是一个关于 JavaScript 引擎的 hello world 应用，你只需在默认程序中，新增四行代码即可：

```
#include <JavaScriptCore/JavaScript.h>

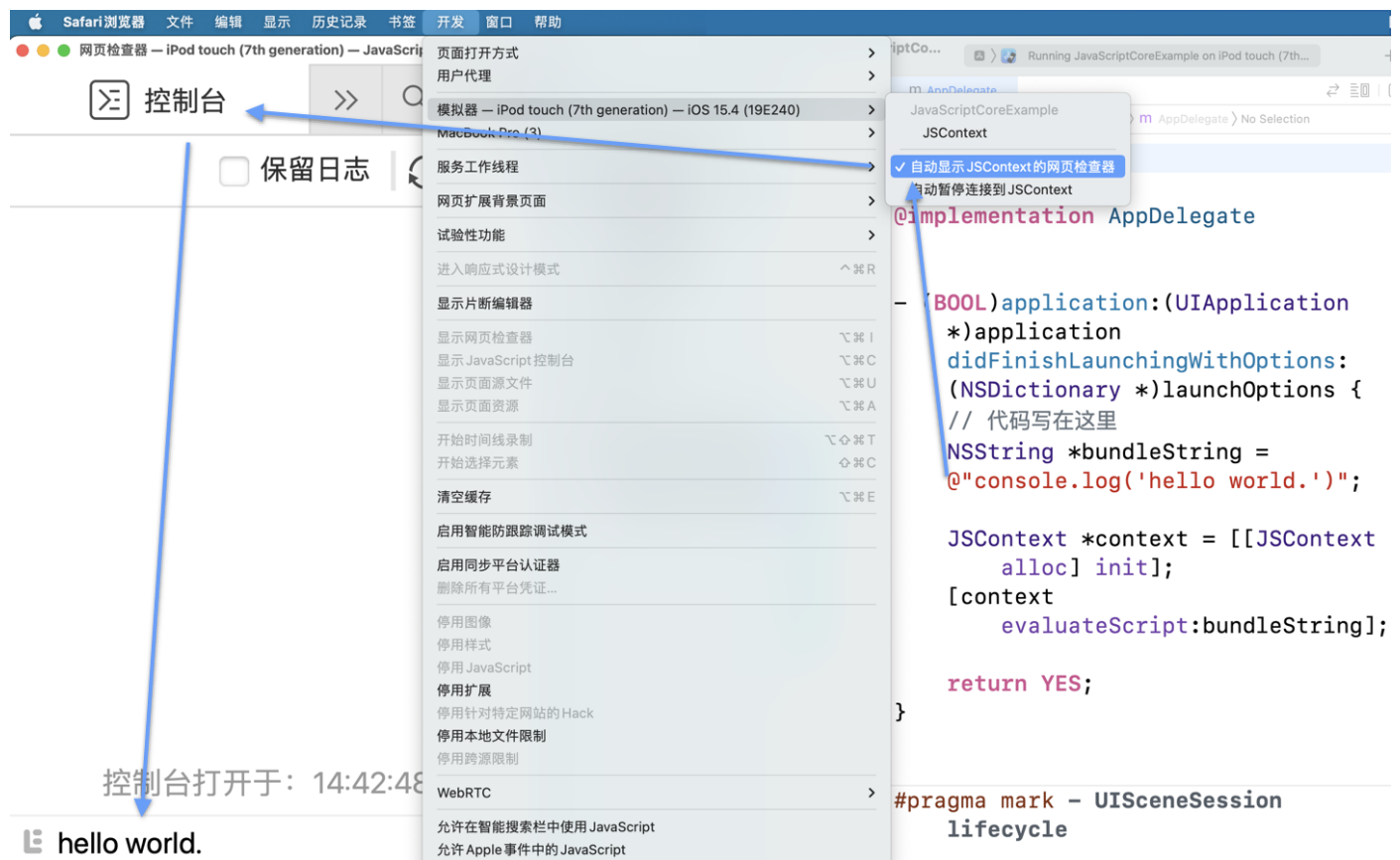
NSString *bundleString = @"console.log('hello world.')";

JSContext *context = [[JSContext alloc] init];

[context evaluateScript:bundleString];
```

⑦当完成代码的新增后，你需要点击 Xcode 左上角的三角形开始按钮，这样，Xcode 就会把这个最简单 App 运行在模拟器中了。

此时，你启动的 JavaScriptCoreExample App，就会打印出 JavaScript 引擎的 hello world 日志，该日志可在 Safari 的控制台看到，截图我放在了下面。



JSContext

现在，hello world 程序跑是跑起来了，虽然只有 4 行代码，而双向通信的秘密就在其中，它就是 JSContext。

那 JSContext 究竟是什么呢？

我们先看下定义，JSContext 在 JavaScript Core 官方文档中定义只有一句话：

```
JSContext:A JavaScript execution environment.
```

JSContext 是 JavaScript 的执行环境。是的，它就是存放 JavaScript 运行过程中，需要用到的各类对象的环境。

```
@interface JSContext : NSObject
```

从上述声明中，你可以看到 JSContext 对象在 Objective-C 中就是个普通的对象。那 JSContext 在 JavaScript 中又是什么呢？

有过 JavaScript 开发经验的同学，肯定接触过 this。在 JavaScript 代码中，this 指的就是 JSContext。咱们可以写个例子演示一下。

```
// JavaScript
var num0 = 0;
this.num0 // 0

// Objective-C
context[@"num0"] //0
```

在上述代码中，咱们先声明了一个 JavaScript 变量，该变量是 num0，它的值是 0。接着我们打印 this.num0 那么结果就是 0。

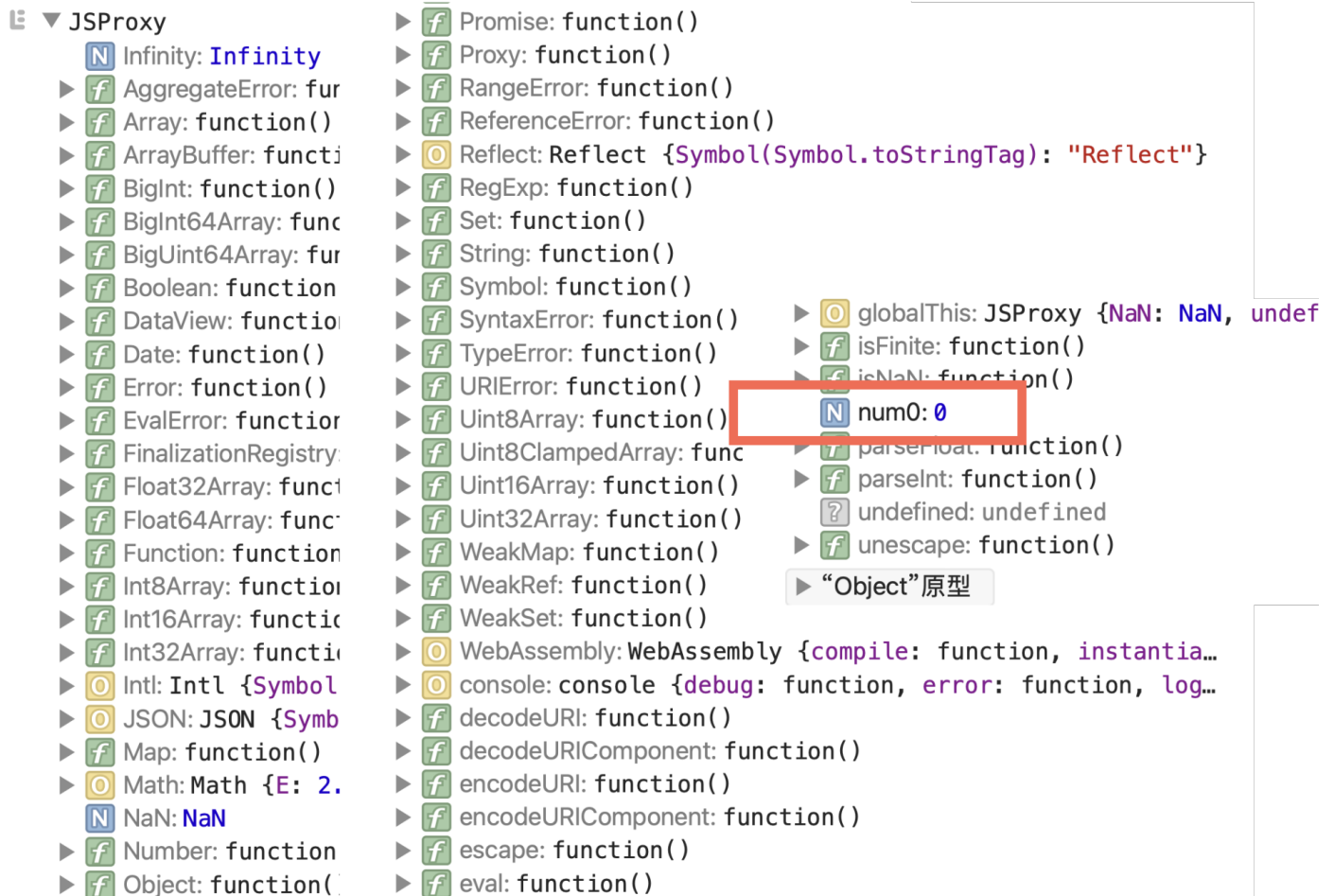
同样，在 Objective-C 中，咱们也可以通过 context[@"num0"] 取到同样的值，打印出来的结果也是 0。

这也就验证了，JavaScript 全局执行环境中的 this，和引擎在 Objective-C 中创建的 context 就是一回事。

如果你好奇，context 究竟长什么样子，你也可以把它打印出来。

```
// JavaScript
console.log(this);
```

我把打印出来的截图，放在了下方：



你可以看到 Safari this 的值，它是一个 JSPoxy 对象，该对象上大概有 30 多个属性，比如数字类型 Number、日期 Date、日志 console 等等，这些属性基本都是 JavaScript 引擎自带的数据类型或方法。

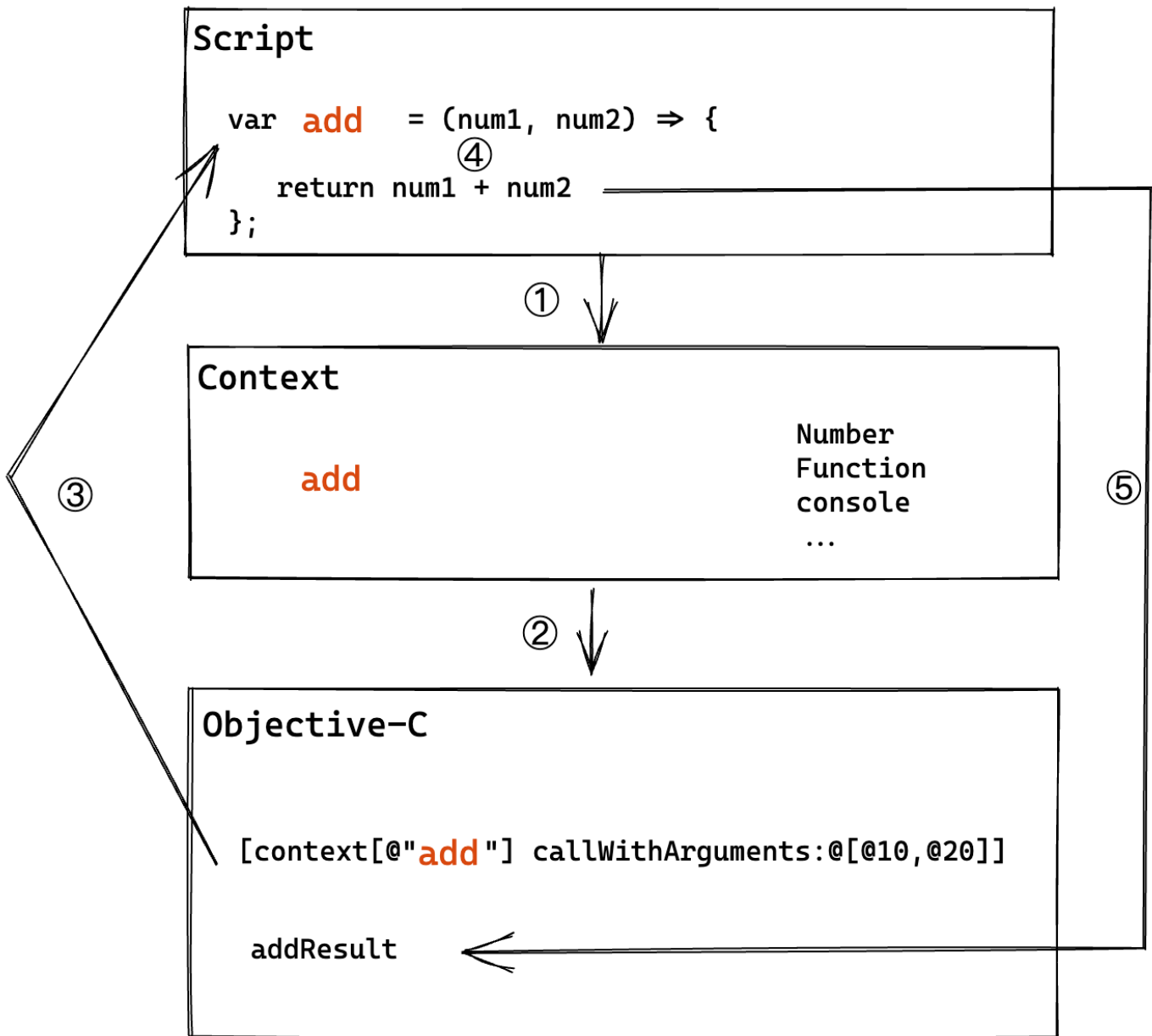
除了引擎自带的属性外，JSPoxy 对象上还有个特殊的属性，是我们自定义的变量 num0。

关键来了，既然我们能在 Objective-C 中创建和使用 JSContext，又能在 JavaScript 获取和使用 JSContext，这不就能打通两个不同语言环境了吗？

是的，JSContext 就是实现双向通讯的基础。

咱们以简单的加法减法为例，深度剖析一下二者的双向通讯的细节。

首先，看下 Objective-C 是如何主动向 JavaScript 传递消息的，我给你画了一张加法函数的执行顺序图：



```
// ①
NSString *bundleString2 =
@"var add = (num1, num2) => {
    // ④
    return num1 + num2
}";
[context evaluateScript:bundleString2];

// ②
JSvarue *addFunction = context[@"add"];
```

```
// ③
JSvarue *addResult = [addFunction callWithArguments:@[@10,@20]];

// ⑤
NSLog(@"10+20:%d", [addResult toInt32]); // 10+20=30
```

首先，当我们使用 context 的 evaluateScript 方法执行 JavaScript 脚本时，引擎会将该脚本中的变量挂载 context 对象的属性上。

这样一来，在 JavaScript 脚本中创建的 add 函数，就能在 Objective-C 中通过 context[@"add"] 获取到了。为了方便后续调用，咱们可以将 JavaScript 函数保存在 Objective-C 的变量 addFunction 中。

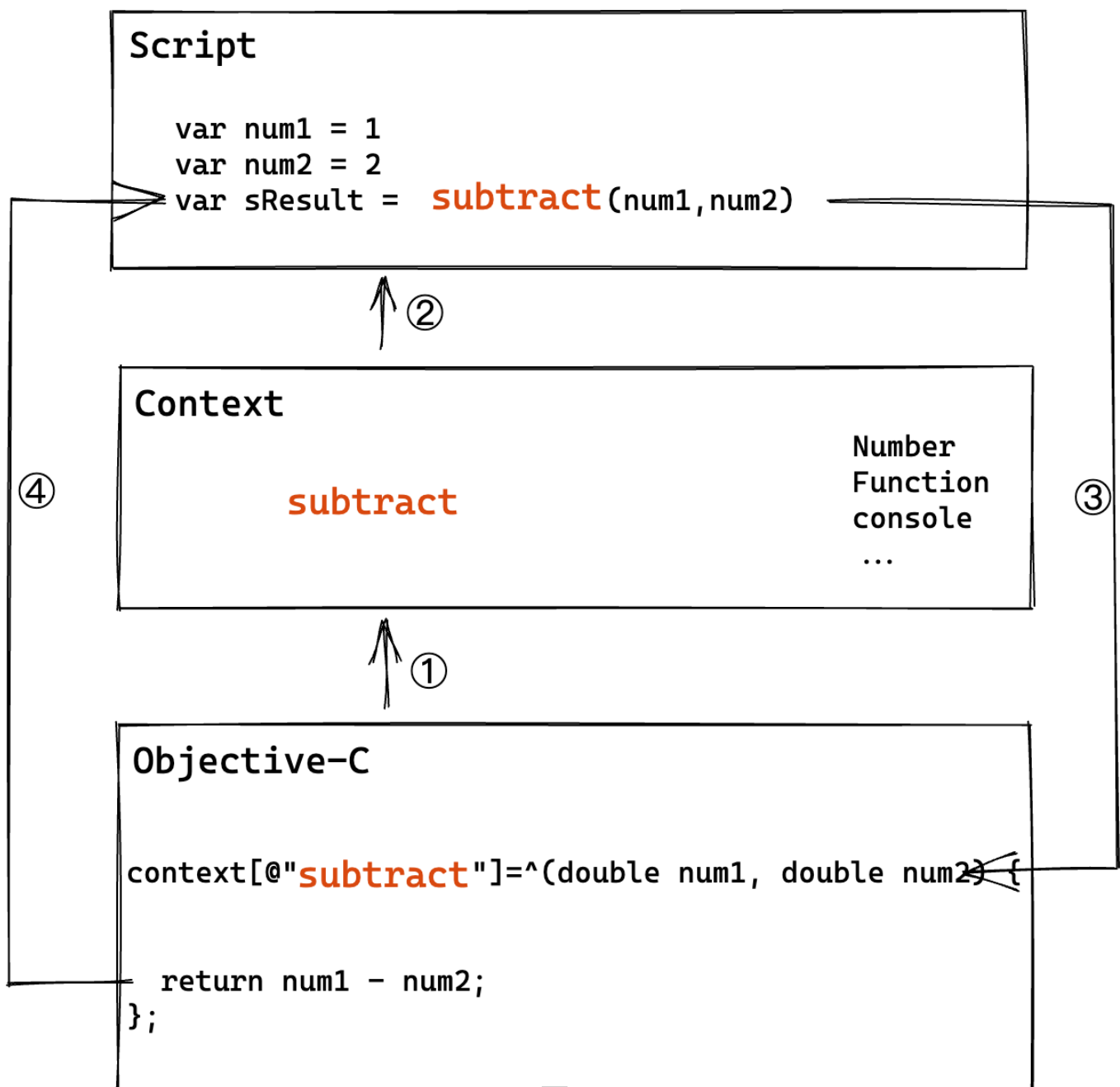
此时，紧接着，咱们就能在 Objective-C 中执行 [addFunction callWithArguments:@[@10,@20]]，并调用并传递参数给 JavaScript 了。

换成 JavaScript 视角，它能够通过函数的入参变量 num1、num2，收到从 Objective-C 中传入的两个值 10 和 20，然后执行加法运行，生成结果 30。

最后，在 JavaScript 中计算出的结果 30，它会返回给 Objective-C 的变量 addResult。以上就是，Objective-C 将 10 和 20 两个数值传给 JavaScript 的完整过程。

我们可以从中延伸思考一下，既然可以传数字，那么也可以传字符串，甚至是数组对象函数，又或者是原生视图节点的引用，这不就是所谓的 Objective-C 向 JavaScript 进行通信吗？

类似地，咱们再来看下 JavaScript 是如何主动向 Objective-C 传递消息的，这次咱们用减法函数来举例，我给你画了一张它的执行顺序图：



```
// ①
context[@"subtract"] = ^(double num1, double num2) {
```

```
    // ③
    return num1 - num2;
};
```

```
// ②
[context evaluateScript:
    @"var subtractResult = subtract(1,2)"
    // ④
    "console.log(subtractResult)"];
```

这时，咱们的第一步是创建一个 Objective-C 的减法函数，并将该减法函数赋值给 `context[@"subtract"]`。

第二步，此时在 JavaScript 中，`this` 就是 `context`，因此 JavaScript 能通过 `this.subtract` 获取到 Objective-C 的减法函数。在实际写代码时，直接使用全局变量 `subtract` 和使用 `this.subtract` 是一回事，因此 `this` 咱们也可以省略不写，直接使用 `subtract(1,2)` 进行调用。

第三步，在 Objective-C 的减法函数调用后，它的变量 `num1`、`num2` 分别会收到从 JavaScript 中传来的数字 1 和数字 2，然后执行减法运算，生成结果 `-1`。

最后，Objective-C 减法函数的执行返回值，会被 JavaScript 的变量 `subtractResult` 接收到。

类似于咱们前面介绍的加法运行，咱们可以通过 Objective-C 函数将 JavaScript 中的数字传过去，那么 JavaScript 中的字符串、数组、对象等数据类型也可以传过去，二者的原理是一样的。

在 React Native 中，用于双向通信的 `nativeFlushQueueImmediate`、`__turboModuleProxy` 和 `nativeFabricUIManager` 三大宿主对象上，挂载了类似咱们加法 `add`、减法 `subtract` 函数，它们都是使用 `JSContext` 实现的。

总结

最后，我们总结一下，React Native 新架构是怎么实现 JavaScript 和 Native 之间的双通信呢？实现的关键是 `JSContext`。

React Native 借助 `JSContext` 实现了宿主对象，这些宿主对象能同时被 JavaScript 和 C++ 操作，因此，在 JavaScript 和 C++ 函数相互调用时，能通过参数的形式，进行跨语言的传参，进而实现了双向通信。

当然，在具体实现上，React Native 新架构借助的是自研的 Hermes 引擎进行实现，而不是咱们今天举例的 JavaScript Core 引擎。

思考题

你知道 Hermes 引擎有哪些特点？React Native 新架构为什么要选择 Hermes 引擎呢？

有什么问题欢迎在评论区留言，我们下一讲见。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

上一篇 29 | 弄清现状：新架构预览版究竟长什么样？

下一篇 31 | 多线程：RN底层是如何保障交互体验的？

精选留言 1